# Agent Swarm Architecture - 4 Maquinas como Equipo de Desarrollo

**Fecha:** 2026-03-21 **Autor:** Cesar / Claude Code **Estado:** Propuesta de arquitectura
**Producto:** Ecosistema BAAT Digital / SuperPago

---

## 1. Vision

Cuatro computadoras trabajando como un equipo de desarrollo coordinado, donde cada maquina:

- Toma tareas pendientes de GitHub automaticamente
- Desarrolla features completos (codigo + tests + push)
- Se comunica con las otras maquinas via DynamoDB
- Respeta dependencias entre tareas
- Comparte learnings y contratos de API
- Opera de forma autonoma durante horas sin intervencion humana

---

## 2. Estado Actual del Sistema

### 2.1 Que ya existe (Agent Memory System)

| Componente | Archivo | Estado |
|---|---|---|
| Atomic locking (DynamoDB) | `scripts/agent_memory/claim_task.py` | Produccion |
| Release + learnings | `scripts/agent_memory/release_task.py` | Produccion |
| Bootstrap (genera CONTEXT.md) | `scripts/agent_memory/bootstrap.py` | Produccion |
| GitHub Board sync | `scripts/agent_memory/sync_github.py` | Produccion |
| Team status dashboard | `scripts/agent_memory/team_status.py` | Produccion |
| Model selector (haiku/sonnet/opus) | `scripts/agent_memory/model_selector.py` | Produccion |
| DynamoDB client (CRUD + locking) | `scripts/agent_memory/dynamo_client.py` | Produccion |
| GitHub client (GraphQL + gh CLI) | `scripts/agent_memory/github_client.py` | Produccion |
| Config (AWS + GitHub IDs) | `scripts/agent_memory/config.py` | Produccion |
| Tabla DynamoDB | `infra/create_agent_memory_table.py` | Produccion |
| Sync cron (cada 15 min) | `.github/workflows/agent_memory_sync.yml` | Produccion |
| Tests (28 tests) | `scripts/agent_memory/tests/` | Produccion |

### 2.2 Tabla DynamoDB: covacha-agent-memory

**Esquema single-table:**

```
PK (Partition Key)        SK (Sort Key)     Contenido
──────────────────────────────────────────────────────────────
TASK#043                  META              number, title, labels, stat
TASK#043                  LOCK              locked_by, locked_at, machi
LEARNING#covacha-payment  patterns          gotchas[], updated_by, upda
TEAM#backend-mac-1        STATUS            team, machine, current_task
SYNC#github               META              last_sync, errors[]
```

**GSIs:** `status-gsi` (queries por estado), `label-gsi` (queries por label) **TTL:** Habilitado en campo `ttl` (auto-limpieza de locks expirados)

### 2.3 Flujo actual (manual)

```
Humano ejecuta:
  1. python bootstrap.py --team backend --machine mac-1 --module covach
  2. Lee CONTEXT.md generado
  3. python claim_task.py --task 043 --team backend --machine mac-1
  4. [Implementa en Claude Code]
  5. python release_task.py --task 043 --status done --learning "..."
  6. python team_status.py --label backend
```

### 2.4 Limitaciones actuales

| Limitacion | Impacto | Solucion propuesta |
|---|---|---|
| Flujo manual (requiere humano) | No escala a 4 maquinas | Agent Loop autonomo |
| Sin dependencias entre tareas | Maquinas trabajan en paralelo sin orden | Prefijo DEP# en DynamoDB |
| Sin mensajes inter-maquina | Islas sin comunicacion | Prefijo MSG# en DynamoDB |
| Sin contratos de API compartidos | Frontend no sabe que endpoints creo backend | Contract Registry |
| Sin heartbeat | Locks muertos por 30 min si maquina se cae | Ping cada 2 min |
| Sync cada 15 min | Latencia alta para coordinacion | Reducir a 5 min |
| Solo 2 maquinas configuradas | Subutilizacion | Escalar a 4 |
| Sesiones cortas (~30 min) | Pierde contexto al compactar | Context Mode plugin |

---

## 3. Arquitectura Propuesta: Agent Swarm

### 3.1 Diagrama de alto nivel

```
┌──────────────────────────────────────────────────────────────────┐
│                      GitHub Project Board                        │
│                 (fuente de verdad: issues + PRs)                 │
│    baatdigital/projects/1 (SuperPago)                            |
```

```
└──────────────────────────────────────────────────────────────┘
                              │
                              │ sync cada 5 min
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                DynamoDB: covacha-agent-memory                 │
│                                                               │
│  TASK#     → metadata de issues (status, labels, branch)      │
│  LOCK#     → locks atomicos (quien trabaja en que)            │
│  DEP#      → dependencias entre tareas          [NUEVO]       │
│  MSG#      → mensajes inter-maquina             [NUEVO]       │
│  CONTRACT#→ contratos de API publicados         [NUEVO]       │
│  LEARNING#→ gotchas acumulados por modulo                     │
│  TEAM#     → status de cada maquina + heartbeat               │
│  SYNC#     → timestamp de ultima sincronizacion               │
└──────────────────────────────────────────────────────────────┘
      │           │           │           │
      │           │           │           │
  ┌───▼───┐   ┌───▼───┐   ┌───▼───┐   ┌───▼───┐
  │ mac-1 │   │ mac-2 │   │ mac-3 │   │ mac-4 │
  │       │   │       │   │       │   │       │
  │BACKEND│   │FRONT  │   │TESTS  │   │  OPS  │
  │       │   │END    │   │+ QA   │   │+ INTEG│
  │ Loop  │   │ Loop  │   │ Loop  │   │ Loop  │
  │  ↕    │   │  ↕    │   │  ↕    │   │  ↕    │
  │Context│   │Context│   │Context│   │Context│
  │ Mode  │   │ Mode  │   │ Mode  │   │ Mode  │
  └───────┘   └───────┘   └───────┘   └───────┘
```

## 3.2 Especializacion por nodo (DINAMICA)

> **IMPORTANTE:** Las maquinas NO estan hardcodeadas. El sistema es auto-discovery. Cada nodo declara sus capabilities en `~/.covacha-node.yml` y se registra automaticamente. Ver documento completo: **AGENT-SWARM-DYNAMIC-NODES.md**

**Ejemplo con 4 nodos (pero funciona con 1 o con 10):**

| Nodo (auto-detect) | Rol (auto-assign) | Repos (auto-detect) | Capabilities |
|---|---|---|---|
| cesar-macbook-pro | tech_lead | covacha-payment, covacha-core, covacha-libs | backend, architecture |
| casp-mac-studio | developer | mf-core, mf-payment, mf-marketing | frontend, ui |
| casp-mac-mini-m4 | tester | todos (read) + tests/ | testing, e2e |
| casp-imac-5k | project_owner | covacha-notification, covacha-projects | ops, cross-repo |

## 3.3 Nuevos prefijos DynamoDB

**DEP# (Dependencias entre tareas)**

```
{
```

```
  "PK": "DEP#044",
  "SK": "DEPENDS_ON#043",
  "blocker_task": "043",
  "blocker_status": "todo",
  "blocked_task": "044",
  "created_at": 1711036800,
  "auto_detected": false
}
```

**Logica:** Una tarea solo es "claimable" si TODOS sus blockers tienen status = done.

**MSG# (Mensajes inter-maquina)**

```
{
  "PK": "MSG#1711036800-mac-1",
  "SK": "BROADCAST",
  "from_machine": "mac-1",
  "from_team": "backend",
  "type": "task_completed",
  "task_id": "043",
  "payload": {
    "endpoint": "POST /api/v1/spei/transfer",
    "contract_id": "CONTRACT#spei-transfer-v1",
    "branch": "feature/ISS-043-spei-transfer",
    "pr_url": "https://github.com/baatdigital/covacha-payment/pull/87"
  },
  "ttl": 1711123200,
  "read_by": ["mac-2", "mac-4"]
}
```

**Tipos de mensaje:**

- `task_completed` — tarea terminada, incluye artefactos
- `task_blocked` — tarea bloqueada, necesita ayuda
- `contract_published` — nuevo contrato de API disponible
- `review_requested` — pide review a otra maquina
- `dependency_resolved` — dependencia resuelta, desbloquea tareas

**CONTRACT# (Contratos de API)**

```
{
  "PK": "CONTRACT#spei-transfer-v1",
  "SK": "META",
  "name": "SPEI Transfer",
  "version": "v1",
  "method": "POST",
  "path": "/api/v1/spei/transfer",
  "request_schema": {
    "clabe_destino": "string (18 digits)",
    "monto": "Decimal",
    "concepto": "string (max 40 chars)",
    "referencia_numerica": "string (7 digits)"
  },
```

```
  "response_schema": {
    "success": "boolean",
    "transaction_id": "string (UUID)",
    "status": "pending|completed|failed",
    "tracking_key": "string"
  },
  "published_by": "mac-1",
  "task_id": "043",
  "created_at": 1711036800,
  "consumed_by": ["044", "045"]
}
```

---

# 4. Agent Loop (Daemon por maquina)

## 4.1 Ciclo de vida

```
┌──────────────────────────────────────────────────┐
│              AGENT LOOP (por maquina)            │
│                                                  │
│   ┌──────────┐                                   │
│   │  START   │                                   │
│   └──────────┘                                   │
│        ▼                                         │
│   ┌──────────┐     no hay tareas                 │
│   │BOOTSTRAP │──────────────────► SLEEP 5min     │
│   │+ SYNC    │                      │            │
│   └──────────┘                      ▼            │
│        ▼                       CHECK MSGS        │
│   ┌──────────┐                 (leer MSG#)       │
│   │  CHECK   │                      │            │
│   │  MSGS    │◄─────────────────────┘            │
│   └──────────┘                                   │
│        ▼                                         │
│   ┌──────────┐     deps no resueltas             │
│   │  FIND    │──────────────────► SLEEP 2min     │
│   │NEXT TASK │                                   │
│   └──────────┘                                   │
│        ▼                                         │
│   ┌──────────┐     lock failed                   │
│   │  CLAIM   │──────────────────► RETRY          │
│   │  TASK    │                                   │
│   └──────────┘                                   │
│        ▼                                         │
│   ┌──────────┐                                   │
│   │IMPLEMENT │ ← Claude Code session             │
│   │  + TEST  │   (con Context Mode)              │
│   └──────────┘                                   │
│        ▼                                         │
│   ┌──────────┐     tests fail                    │
│   │  PUSH    │──────────────────► FIX + RETRY    │
│   │  CODE    │                                   │
```

```
│   └──────┬──────┘                                  │
│          ▼                                          │
│   ┌──────────────┐                                  │
│   │  RELEASE     │                                  │
│   │+ PUBLISH     │ ← msg + contract + learnings     │
│   └──────┬──────┘                                  │
│          ▼                                          │
│   ┌──────────────┐                                  │
│   │HEARTBEAT     │──────► GOTO BOOTSTRAP            │
│   └──────────────┘                                  │
└─────────────────────────────────────────────────────┘
```

## 4.2 Pseudocodigo del Agent Loop

```
# scripts/agent_memory/agent_loop.py

def agent_loop(team: str, machine: str, repos: list[str]):
    """Loop autonomo que corre en cada maquina."""

    while True:
        # 1. Heartbeat
        update_heartbeat(team, machine)

        # 2. Sync GitHub (si hace >5 min del ultimo sync)
        if should_sync():
            sync_github()

        # 3. Leer mensajes pendientes
        messages = get_unread_messages(machine)
        for msg in messages:
            process_message(msg, team, machine)
            mark_as_read(msg, machine)

        # 4. Encontrar siguiente tarea
        task = find_next_task(
            team=team,
            machine=machine,
            repos=repos,
            check_dependencies=True  # Solo tareas con deps resueltas
        )

        if not task:
            log(f"{machine}: No hay tareas disponibles. Durmiendo 5 min
            sleep(300)
            continue

        # 5. Reclamar tarea (atomico)
        claimed = claim_task(task["number"], team, machine)
        if not claimed:
            log(f"{machine}: Lock collision en ISS-{task['number']}. Re
            continue

        # 6. Implementar (delega a Claude Code)
```

```
        result = implement_task(
            task=task,
            machine=machine,
            model=task["recommended_model"],
            context_mode=True  # Habilita Context Mode para sesiones la
        )

        # 7. Release + publish
        release_task(
            task_id=task["number"],
            team=team,
            machine=machine,
            status=result["status"],  # done │ blocked
            learnings=result.get("learnings", [])
        )

        # 8. Publicar contrato si es backend endpoint
        if result.get("contract"):
            publish_contract(result["contract"])

        # 9. Enviar mensaje broadcast
        send_message(
            from_machine=machine,
            type="task_completed" if result["status"] == "done" else "t
            task_id=task["number"],
            payload=result.get("artifacts", {})
        )

        # 10. Desbloquear tareas dependientes
        if result["status"] == "done":
            unblock_dependents(task["number"])


def find_next_task(team, machine, repos, check_dependencies=True):
    """Encuentra la siguiente tarea disponible respetando prioridad y d

    available = get_available_tasks(label=team, status="todo")

    for task in sorted(available, key=priority_score):
        # Filtrar por repos de esta maquina
        if task["repo"] not in repos:
            continue

        # Verificar dependencias
        if check_dependencies:
            deps = get_dependencies(task["number"])
            if any(d["blocker_status"] != "done" for d in deps):
                continue

        # Verificar que no este lockeada
        if is_locked(task["number"]):
            continue
```

```
        return task

    return None


def implement_task(task, machine, model, context_mode=True):
    """Lanza una sesion de Claude Code para implementar la tarea."""

    repo_path = get_repo_path(task["repo"])
    branch = task.get("branch") or f"feature/ISS-{task['number']}"

    # Generar prompt para Claude Code
    prompt = generate_implementation_prompt(
        task=task,
        contracts=get_relevant_contracts(task),
        learnings=get_learnings(task["repo"]),
        messages=get_recent_messages(limit=10)
    )

    # Ejecutar Claude Code con --print (no-interactive) o via API
    result = run_claude_code(
        prompt=prompt,
        cwd=repo_path,
        model=model,
        max_turns=50,
        context_mode=context_mode
    )

    return result
```

### 4.3 Heartbeat

```
def update_heartbeat(team: str, machine: str):
    """Actualiza last_seen para detectar maquinas caidas."""
    table.update_item(
        Key={"PK": f"TEAM#{team}-{machine}", "SK": "STATUS"},
        UpdateExpression="SET last_seen = :now, heartbeat_count = heart
        ExpressionAttributeValues={
            ":now": int(time.time()),
            ":one": 1
        }
    )

def check_stale_machines():
    """Libera locks de maquinas que no han hecho heartbeat en 5 min."""
    teams = get_all_team_statuses()
    now = int(time.time())

    for team in teams:
        if team.get("current_task") and (now - team.get("last_seen", 0)
            log(f"STALE: {team['machine']} no responde. Liberando ISS-{
            release_task(team["current_task"], status="todo")  # Re-enc
            send_message(
```

```
                from_machine="dispatcher",
                type="machine_stale",
                payload={"machine": team["machine"], "task": team["curr
            )
```

---

# 5. Flujo Cross-Repo Completo (Ejemplo)

## Escenario: EP-SP-019 "Transferencias SPEI"

```
GitHub Board:
  ISS-043: [backend] Endpoint POST /api/v1/spei/transfer
  ISS-044: [frontend] UI formulario transferencia (depends-on: 043)
  ISS-045: [test] E2E transferencia completa (depends-on: 043, 044)
  ISS-046: [docs] Documentacion API SPEI (depends-on: 043)
```

## Timeline

```
T=0min   Dispatcher evalua board
         → mac-1 ← ISS-043 (backend, sin deps)
         → mac-4 ← ISS-046 (docs, puede empezar estructura)
         → mac-2 IDLE (espera ISS-043)
         → mac-3 IDLE (espera ISS-043 + ISS-044)

T=5min   mac-1: claim ISS-043, crea branch feature/ISS-043-spei-transfe
         mac-4: claim ISS-046, crea estructura docs base

T=45min  mac-1: DONE ISS-043
         → Publica CONTRACT#spei-transfer-v1 en DynamoDB
         → MSG: "ISS-043 done, contract available"
         → DEP#044.blocker_status = done
         → DEP#045.blocker_043_status = done

T=46min  mac-2: detecta ISS-044 desbloqueada
         → Lee CONTRACT#spei-transfer-v1
         → claim ISS-044, crea branch feature/ISS-044-transfer-ui

T=47min  mac-1: busca siguiente tarea (ya termino ISS-043)
         → Encuentra ISS-048 (otro backend task)
         → claim ISS-048, sigue trabajando

T=90min  mac-2: DONE ISS-044
         → MSG: "ISS-044 done, component: TransferFormComponent"
         → DEP#045.blocker_044_status = done

T=91min  mac-3: detecta ISS-045 desbloqueada (043 + 044 done)
         → claim ISS-045
         → Lee contratos + componentes de 043 y 044
         → Escribe tests E2E

T=120min mac-3: DONE ISS-045
         → Toda la epica ISS-043,044,045,046 completada
```

# 6. Context Mode - Integracion

## 6.1 Que es

Context Mode es un servidor MCP + sistema de hooks que resuelve dos problemas por maquina:

1. **Ahorro de contexto** — Ejecuta comandos en sandbox aislado. Solo stdout entra al contexto.
    - 315 KB raw → 5.4 KB procesado (98% reduccion)
2. **Continuidad de sesion** — Trackea eventos en SQLite local + FTS5 search.
    - Cuando Claude Code compacta, reconstruye estado completo automaticamente.

## 6.2 Herramientas MCP

| Tool | Que hace | Ahorro |
|---|---|---|
| `ctx_batch_execute` | Multiples comandos en 1 llamada | 986 KB → 62 KB |
| `ctx_execute` | Ejecuta codigo (11 lenguajes), solo stdout | 56 KB → 299 B |
| `ctx_execute_file` | Procesa archivos en sandbox | 45 KB → 155 B |
| `ctx_index` | Chunk markdown en FTS5 + BM25 | 60 KB → 40 B |
| `ctx_search` | Query sobre contenido indexado | On-demand |
| `ctx_fetch_and_index` | Fetch URL + chunk + index | 60 KB → 40 B |

## 6.3 Session Continuity

Hooks que capturan eventos:

| Hook | Que captura |
|---|---|
| PostToolUse | File edits, git ops, tasks, errors, MCP tools |
| UserPromptSubmit | Decisiones del usuario, correcciones, preferencias |
| PreCompact | Snapshot <=2 KB antes de compactar (priority-tiered) |
| SessionStart | Restaura estado despues de compactar o --continue |

**Resultado:** Sesiones de 30 min → 3+ horas sin perder el hilo.

## 6.4 Instalacion (por maquina)

```
# Claude Code plugin (automatico)
/plugin marketplace add mksglu/context-mode
/plugin install context-mode@context-mode

# Verificar
/context-mode:ctx-doctor
```

```
# Cursor (manual, para las maquinas que usen Cursor)
npm install -g context-mode
mkdir -p .cursor/rules
cp node_modules/context-mode/configs/cursor/context-mode.mdc .cursor/ru
```

### 6.5 Relacion con Agent Memory

```
Context Mode = capa INTRA-maquina (sesiones largas, ahorro de tokens)
Agent Memory = capa INTER-maquina (coordinacion, locks, mensajes)

Cada maquina tiene:
  ┌──────────────────────────────┐
  │     Claude Code Session      │
  │  ┌────────────────────────┐  │
  │  │    Context Mode        │  │  ← Sesion larga sin perder contexto
  │  │  (SQLite + FTS5 local) │  │
  │  └────────────────────────┘  │
  │  ┌────────────────────────┐  │
  │  │    Agent Memory        │  │  ← Coordinacion con otras maquinas
  │  │  (DynamoDB remoto)     │  │
  │  └────────────────────────┘  │
  │  ┌────────────────────────┐  │
  │  │    Agent Loop          │  │  ← Daemon autonomo
  │  │  (claim/implement/     │  │
  │  │   release/repeat)      │  │
  │  └────────────────────────┘  │
  └──────────────────────────────┘
```

---

# 7. Plan de Implementacion

### Fase 1: Foundation (1-2 dias)

**Objetivo:** Escalar infraestructura de 2 a 4 maquinas.

| Tarea | Archivo | Cambio |
|---|---|---|
| Agregar mac-3, mac-4 a config | `config.py` | MACHINES = ["mac-1","mac-2","mac-3","mac-4"] |
| Agregar especializacion por maquina | `config.py` | MACHINE_SPECIALIZATION = {...} |
| Reducir sync interval a 5 min | `agent_memory_sync.yml` | cron: '*/5 * * * *' |
| Agregar campo heartbeat a team status | `dynamo_client.py` | heartbeat_count, last_seen |
| Instalar Context Mode en 4 maquinas | Manual | /plugin marketplace add |
| Tests para nuevas maquinas | `tests/` | test_4_machines_no_collision |

## Fase 2: Dependencies + Messages (2-3 dias)

**Objetivo:** Maquinas se comunican y respetan orden de tareas.

| Tarea | Archivo | Cambio |
|---|---|---|
| Crear prefijo DEP# en DynamoDB | `dynamo_client.py` | add_dependency(), get_dependencies(), resolve_dependency() |
| Crear prefijo MSG# en DynamoDB | `dynamo_client.py` | send_message(), get_unread_messages(), mark_as_read() |
| Parsear label `depends-on:ISS-XXX` | `sync_github.py` | Extraer deps de labels al sincronizar |
| CLI: agregar dependencia | `add_dependency.py` | --task 044 --depends-on 043 |
| CLI: ver mensajes | `check_messages.py` | --machine mac-2 |
| find_next_task con deps | `dynamo_client.py` | Filtrar tareas con deps no resueltas |
| Tests | `tests/` | test_dependency_blocking, test_messages_read_unread |

## Fase 3: Contract Registry (1-2 dias)

**Objetivo:** Backend publica contratos, frontend los consume.

| Tarea | Archivo | Cambio |
|---|---|---|
| Crear prefijo CONTRACT# | `dynamo_client.py` | publish_contract(), get_contract(), get_contracts_for_task() |
| CLI: publicar contrato | `publish_contract.py` | --name "SPEI Transfer" --method POST --path /api/v1/... |
| Integrar en release_task | `release_task.py` | Opcion --contract para publicar al liberar |
| Auto-deteccion de contratos (Flask routes) | `contract_detector.py` | Parsea Flask blueprints → genera contrato |
| Tests | `tests/` | test_contract_publish_consume |

## Fase 4: Agent Loop (3-5 dias)

**Objetivo:** Cada maquina corre autonomamente.

| Tarea | Archivo | Cambio |
|---|---|---|
| Agent Loop daemon | `agent_loop.py` | Loop infinito: bootstrap→find→claim→implement→release |
| Heartbeat checker | `heartbeat_checker.py` | Detecta maquinas stale, libera locks |
| Integration con Claude Code --print | `agent_loop.py` | Lanza sesion no-interactiva |
| Generador de prompts por tipo de tarea | `prompt_generator.py` | Genera prompt con context, contracts, learnings |

| | | | |
|---|---|---|---|
| Dos maquinas intentan misma tarea | Media | Bajo | Atomic locking ya resuelve esto |
| Maquina se cae mid-task | Media | Medio | Heartbeat + auto-release + re-encolar |
| Dependencia circular | Baja | Alto | Validacion al crear DEP#, detectar ciclos |
| Claude Code falla mid-implementation | Media | Medio | Agent Loop retry con backoff |
| Contrato de API cambia post-publicacion | Media | Alto | Versionado en CONTRACT# (v1, v2) |
| Context window se llena pese a Context Mode | Baja | Medio | Split en sub-tareas mas pequenas |
| GitHub API rate limit | Baja | Medio | Cache en DynamoDB, sync batch |
| Lock TTL expira en tarea larga | Media | Medio | Heartbeat renueva TTL cada 2 min |

---

# 10. Metricas de Exito

| Metrica | Baseline (hoy) | Target |
|---|---|---|
| Tareas completadas/dia | 2-3 (manual) | 10-15 (autonomo) |
| Tiempo promedio por tarea | 2-4 horas | 30-60 min |
| Colisiones entre maquinas | N/A | <2% |
| Uptime de maquinas | Manual | >95% |
| Sesiones sin perder contexto | ~30 min | 3+ horas |
| Features cross-repo coordinados | 0 | 5-8/semana |
| Learnings acumulados/semana | ~5 | 30-50 |

---

# 11. Prerequisitos

## Hardware

- 4 Macs con Claude Code instalado
- Acceso SSH entre maquinas (opcional, para monitoring)
- AWS credentials configuradas en las 4

## Software (por maquina)

- Claude Code v1.0.33+ (para Context Mode plugin)
- Python 3.9+ con boto3, requests
- gh CLI autenticado (gh auth login)
- Node.js 18+ (para Context Mode)
- Git configurado con SSH keys

## AWS

- Tabla DynamoDB `covacha-agent-memory` (ya existe)

- IAM user/role con permisos DynamoDB (ya configurado)
- Opcional: Lambda para dispatcher (Fase 5)

## GitHub

- Issues creados en el board con labels correctos
- Labels `depends-on:ISS-XXX` para dependencias
- Labels de especializacion: backend, frontend, test, docs, cross-repo